

МАНИФЕСТ

NixOS: Неизбежное Будущее

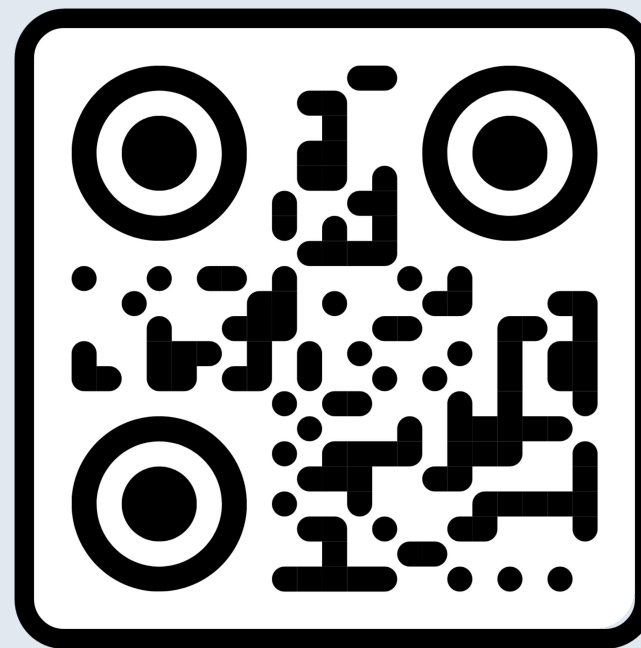
Зачем менять ОС, и почему раз попробовав — вы уже не сможете
остановиться.

Почему меня стоит слушать?

🎓 **Мне 20 лет.** Получил степень BSCS США в 19 лет.

💼 **Ведущий инженер в 18 лет.** Системное и программное обеспечение в квази-гос секторе.

🔗 **Инди-разработчик.** Автор PAM Bio (авторизация на ПК по отпечатку с телефона).



Мои контакты и проекты

ПРЕДЫСТОРИЯ

Короткий путь дистри-хоппинга.

Всё началось на работе. Я поймал себя на мысли, что использую WSL чаще, чем саму Windows.

- **Осознание.** PyCharm, Android Studio и код прекрасно работают на Linux.
- **Ubuntu.** Работает, но некрасиво. Эстетика ближе к MacOS, но не дотягивает.
- **KDE (Kubuntu).** Смена окружения ради визуального комфорта.

Dependency Hell.

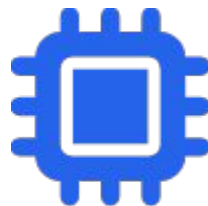
Желание перекатиться на Linux полностью разбилося о реальность установки специфичного софта.

На Ubuntu: Часы боли

Попытки завести DaVinci Resolve, Steam, Proton и Wine превращались в часы чтения форумов и правки конфигов.

На NixOS: Одна строка

DaVinci Resolve встал без танцев с бубном (Fedora не справилась). Steam и Proton заработали мгновенно.



Проклятие CUDA.

Каторга машинного обучения на Linux. Разные проекты требуют разных версий CUDA и конфликтующих ИИ-утилит.

Глобальная установка неизбежно превращает ОС в мусорку.

АНАЛОГИЯ 1

Как Snap, но на уровне всей ОС.

Каждый пакет лежит в изолированной директории в [/nix/store](#) с уникальным хэшем.

Никаких конфликтов глобальных библиотек.

Изоляция без дублирования общих зависимостей.





АНАЛОГИЯ 2

Как `project.toml`, но для системы.

Все зависимости, программы и настройки описаны явно в конфигурационном файле.

Если программы нет в конфиге — её нет в системе. Абсолютная 100% предсказуемость.

АНАЛОГИЯ 3

Как Docker Compose на голом железе.

Декларативное управление сервисами, портами, базами данных и ядром.

Воспроизводимость инфраструктуры без накладных расходов виртуализации.



Больше, чем сумма технологий.

Ни одно сравнение не передаёт весь смысл NixOS.

Это единый, изящный механизм.

Код вместо кликов.

Вам не нужно гадать, "что делает этот кусок кода". Конфиг прямо говорит системе, что должно быть установлено.

Из реального конфига (dev-env.nix):

```
{ pkgs, ... }:  
{  
  environment.systemPackages = with pkgs; [  
    (python3.withPackages (ps: [ ps.tkinter ]))  
    gcc  
    gnumake  
    go  
    google-cloud-sdk  
    nodejs  
    poetry  
  ];  
}
```



КИЛЛЕР-ФИЧА

Генерации. Бесстрашные обновления.

Из коробки. Без настройки BTRFS или LVM
снапшотов.

Захотел сменить ядро? DE? Удалить 100
пакетов за итерацию? Делаете смело, зная, что
всегда есть шаг назад.

”Я случайно удалил полсистемы”.

Катастрофа

Офисный ПК. Чистка конфига. Огромный красный список удаления в терминале. Снесено всё, включая графическую оболочку KDE. Паника как при удалении production БД.



Решение (1 секунда)

Перезагрузка. Выбор предыдущей генерации в меню загрузчика. Система загружается, как ни в чём ни бывало. Продолжение работы.

АРХИТЕКТУРА

”Обновление конфига равнозначно переустановке.”

Так пишут в Wikipedia. Это не всегда технически точно, но будем честны:
NixOS подобрался к этому определению неописуемо ближе, чем любой
другой дистрибутив.

РАЗВЕНЧАНИЕ МИФОВ

Сколько это весит?

Кажется, каждая программа тащит свои копии библиотек? Нет. В NixOS строгая динамическая линковка. **Общие компоненты — общие.**

Реальное потребление диска.

115_{GB}

Heavy Workstation

Все поколения за 30 дней (~100 сборок).

Активные тесты.

50_{GB}

Текущий слепок

Ровно одно актуальное поколение со всеми

тяжелым софтом.

35_{GB}

Мой ноутбук

не захламлен, мало поколений. Объем на

уровне чистой Windows.

Зоопарк зависимостей.

В компаниях или домашних пет-проектах неизбежно наступает хаос.

- ✖ Разные версии CUDA для разных ИИ.
- </> Специфичные версии Python и Node.js.
- 💣 Конфликтующие системные утилиты.



Локальный порядок.

```
nix develop
```

Куда проще и охотней прописать всё
нужное только под один конкретный
проект в его [flake.nix](#).

**Вышли из директории — система снова
чистая.**

Это не работа ради работы.

Писать flake под каждый скрипт не обязательно. Для 80% задач мне хватает глобально установленных Go, Poetry и Python.

Но когда нужен сложный проект — лучше решения просто нет.

КОЧЕВНОСТЬ

Перманентная готовность к форс- мажорам.

Сгорел диск? Срочная командировка?

Новый офис?



Снова "дома" за 15 минут.



Флешка с Ventoy

Монтирование раздела данных.



4 команды

Применение нужного профиля
(например, laptop).



Полная готовность

Окружение, утилиты, пароли и
профиль восстановлены 1:1.

ОДИН РЕПОЗИТОРИЙ

Вся жизнь в одном Flake.

Перетаскивание пакетов с ПК на ноутбук — нетривиальная задача для обычных ОС.

Здесь всё лежит в едином репозитории.

Меняешь профиль в загрузчике — получаешь специфичную для железа сборку с общим ядром твоих программ.

```
outputs = {  
    self,  
    nixpkgs,  
    ...  
}: {  
    nixosConfigurations = {  
        home-pc = mkHost "home-pc" [];  
        work-pc = mkHost "work-pc" [];  
        laptop = mkHost "laptop" [];  
    };  
};
```

Ротация стабильных веток.

Всё как у всех уважающих себя дистрибутивов, только просто, красиво и контролируемо.

Stable

Мажорный апдейт раз в полгода. Боты оперативно тащат только security-бэкпорты. Абсолютная предсказуемость.

Unstable

Rolling updates. Здоровая практика — держать базу на Stable, а отдельный функционал тащить bleeding-edge.

NixOS распространённой, чем вы думаете.

Тихий, но уверенный выбор крупных инфраструктур и серьёзных DevOps-инженеров.

Google, Replit, Target, Shopify и другие.

ПАКЕТНЫЙ МЕНЕДЖЕР

Nix > apt

Попробуйте их пакетный менеджер на WSL или MacOS.

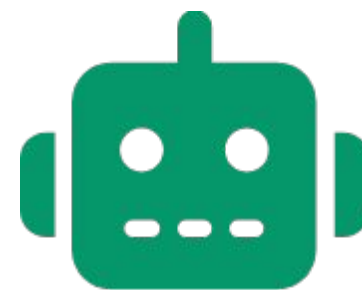
Вы не захотите возвращаться на apt.

СМЕНА ПАРАДИГМЫ

Первая AI prompt-oriented ОС.

Пугает язык конфигурации Nix? С современными нейросетями это незначительный фактор.

Синтаксис больше не барьер. Опишите
желаемое состояние текстом, ИИ выдаст
готовый декларативный блок.



Сообщество и нейросети решают всё.



Мгновенный рефакторинг

Конфиг превратился в спагетти после тестов? ИИ разжёвывает и переписывает его до идеала за 2 минуты.



Copy-Paste культура

Копировать чужие изящные решения с GitHub — стандартная практика, как с i3 или Hyprland.



Стереотип: Gentoo-мазохизм.



Реальность: Выбор инженера.

Человек на NixOS точно знает, что хочет
видеть в системе. От разработчиков
разработчикам.

К этому нужно быть готовым.

Это не Windows. Здесь не всё делается в два клика.

Если что-то не устанавливается при первой попытке — это не повод сходить с ума.

Вам придётся переучиваться, мучать ИИ и Google.

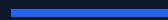
Это нормально при переходе на новую фундаментальную парадигму. Не ожидайте идентичного пользовательского опыта.

Будущее неизбежно.

Dual-boot, WSL или просто установка пакетного менеджера —
начните с малого.

Это заразно.

Готов ответить на вопросы.



NixOS.kz